



Universidad de Jaén

Tema 3 *Frameworks de Programación en el Servidor:* *Spring MVC*

Desarrollo de Aplicaciones Web

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

Este obra está bajo una [Licencia Creative Commons](#)
Atribución-CompartirIgual 4.0 Internacional.

v 3.0.3

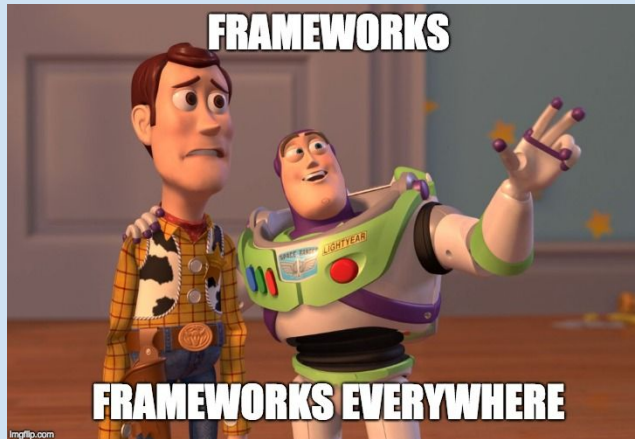
Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Objetivos

- Comprender el concepto de **Framework de programación**, su composición y sus ventajas o posibles inconvenientes
- Conocer algunas **tendencias y tipos de Frameworks de programación web en Java**
- Conocer las **características básicas del framework Spring** y saber utilizarlo en una aplicación web JEE usando **SpringBoot**
- Conocer el mecanismo básico de Inversión de control e **Inyección de dependencias** en Spring
- Conocer las **características de SpringMVC** como Framework de desarrollo web en la plataforma Java
- Saber crear **controladores web con SpringMVC**
- Conocer el mecanismo **selección de vistas JSP** en SpringMVC
- Saber cómo **procesar y validar información de formularios** con SpringMVC
- Saber **configurar JPA** para su uso con SpringBoot





Frameworks para desarrollo de aplicaciones web en Java

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Frameworks de desarrollo software

- **Definición:** Un framework de desarrollo software es una colección de software con un comportamiento *genérico* que puede ser personalizado para realizar un comportamiento *específico*
- **Objetivo:** Aunar principios de diseño, patrones y buenas prácticas en la implementación de aplicaciones con características o necesidades similares
- **Composición:**
 - Bibliotecas de código
 - API
 - Documentación
 - Utilidades de apoyo
 - Pueden ser modulares e incluso permitir la interacción entre varios de ellos



Ventajas e inconvenientes

• Ventajas

- Una vez conocido el framework, el proceso de desarrollo es mucho más **rápido**
- Aplicación de **buenas prácticas** de diseño
- **Reutilización** de código para tareas habituales
- **Soporte** de la comunidad o comercial

• Posibles Inconvenientes

- Proceso de **aprendizaje lento**
 - Excesivo **acoplamiento** de la aplicación. Dificultad para migrar a otros *frameworks* (*lock-in*)
 - **Dependencia de terceros** para corrección de errores, actualizaciones o mejoras
 - **Depuración compleja**
 - Lentitud en la ejecución
- No obstante, un **correcto uso del framework** junto con un buen diseño de la aplicación pueden minimizar los posibles inconvenientes

5

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Frameworks de desarrollo web JVM en el servidor

• Push-Based (o action-based)

- Enrutamiento URLs a acciones (controlador) que “envían” los datos a las vistas
- Struts, [Spring MVC](#), [JAX-RS](#), [Jakarta MVC](#), Stripes, Click, Spark, Vert.x-web

• Pull-Based (o component based)

- Enrutamiento URLs a vistas que “extraen” los datos de controladores
- Arquitecturas basadas en MVP o MVVM
- Interfaces de usuario avanzadas (RIA)
- Curva de aprendizaje superior
- Struts 2, [Jakarta Faces](#) (Mojarra (RI), RichFaces, PrimeFaces, IceFaces, Bootsfaces), Tapestry, Wicket, JBoss Seam, Google Web Toolkit (GWT), SmartGWT, GXT, Vaadin, Errai, ...

• Híbridos (ambos enfoques), Play, ZK, ...

- Pueden ser frameworks independientes para desarrollo web o formar parte de frameworks generalistas

Fuente: http://en.wikipedia.org/wiki/Comparison_of_web_application_frameworks#Java

6

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025





Spring MVC

Framework para aplicaciones web con Spring



Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Spring Framework



- **Framework OpenSource** para simplificar el desarrollo de aplicaciones JVM
 - “Spring handles the infrastructure so you can focus on your application.” Spring Reference Documentation
- **Historia:**
 - 2002, Rod Johnson. 2004, v1.0. Versión actual: 6 (soporte Jakarta EE 10)
 - Desarrollado por VMware <http://spring.io/>, licencia Apache 2.0
- **Contenidos**
 - Modelo de componentes y conjunto de APIs que *aíslan* al programador de los errores y complejidad del *código recurrente para realizar aplicaciones habituales (boilerplate)*
 - Aplicaciones basadas en POJOs sin necesidad de introducir dependencias con la arquitectura usada, e.g. Servlets, EJB, etc.
 - **Reutiliza varios de los módulos de JakartaEE**, e.g. Servlet, JSTL, JSP, JPA, etc.
 - Otras bibliotecas Java de uso habitual
 - **Organizado en módulos:**
 - [Container](#), Data Access/Integration ([SpringData](#), SpringJDBC ...), [SpringMVC](#), AOP, Testing,



Spring Boot



- Framework "*opinionated*" para creación de aplicaciones Spring:
 - Desarrollo de **aplicaciones Java** basadas en Spring
 - Basado en "**convención**" frente a **configuración**: añade configuraciones habituales en aplicaciones Spring
 - Utiliza una selección de **bibliotecas de terceros** y **módulos spring** para resolver problemas habituales, e.g. microservicios
- Para **desarrollo de aplicaciones web**
 - **Versiones .jar** para ejecución en línea
 - Permite lanzar un **contenedor de servlets embebido**, e.g. tomcat, jetty, etc.
 - **Versiones .war** para ejecución en servidores JEE (detalles en apéndice)

Estudiaremos este enfoque

9

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Inicialización de un proyecto Spring Boot con soporte JSP

- <https://start.spring.io/> (o con asistente de IntelliJ)
 - Seleccionar tipo de proyecto maven
 - Añadiremos los starters necesarios: Spring Web, Spring Data y Spring Validation, junto a **módulos Jakarta EE** específicos:
- **Ajustes adicionales**
 - Utilizar un **empaquetado .war** para soporte JSP
 - Indicar módulo de procesamiento de ficheros JSP en Tomcat: Jasper
 - Crear carpeta **src/main/webapp** para contenidos JSP, e.g. /WEB-INF/web.xml

Spring Web **WEB**
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA **SQL**
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Validation **IO**
Bean Validation with Hibernate validator.

```
<packaging>war</packaging> <!-- needed for JSP support -->
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.3</version>
</parent>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency> <!-- needed for JSP support -->
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
</dependencies>
```

```
<!-- IntelliJ JSTL tag autocomplete and fmt module support-->
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>3.0.2</version>
</dependency>
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>3.0.1</version>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

10

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Instanciación de componentes

- Spring soporta **Inversión de Control** para la gestión del ciclo de vida de los componentes, **beans**.
- **Modos de configuración de componentes**
 - Mediante **XML**, e.g. [/WEB-INF/applicationContext.xml](#)
 - **Anotaciones** en POJOs
 - Clases (java) con **código** de configuración (Spring +3.0), e.g.

```
@Configuration
class ClubConfig { ...
    @Bean
    public DataSource getDataSource() { //Create and config DataSource }
```

- **Anotaciones de componentes**

- Uso general: [@Component](#)
- Según su uso en la aplicación
 - [@Controller](#), capa de presentación, e.g. controlador
 - [@Repository](#), capa de persistencia, e.g. DAO
 - [@Service](#), capa de servicio

11

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025

Ámbitos de persistencia de objetos en SpringMVC

- Por defecto todos los **beans** se tratan como **Singletons** (una única instancia en la app)
 - Pueden modificarse con [@Scope\("ámbito"\)](#)
 - e.g. [@Scope\("prototype"\)](#), una instancia por solicitud
- **En aplicaciones Web**, además se pueden definir ámbitos específicos habituales:
 - [@RequestScope](#) (no confundir con [@RequestScoped](#) en JEE)
 - [@SessionScope](#)
 - [@ApplicationScope](#)

- **Ejemplo**

```
@Component
@SessionScope
class PreferenciasUsuario implement Serializable {
}
```

12

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025

Inyección de dependencias en Spring

- Spring dispone de un **mecanismo propio** de inyección de dependencias
- Similar a inyección de dependencias en CDI
 - **Anotaciones** `@Autowired` o `@Inject`
 - En atributos de componentes o en métodos
 - Los componentes pueden definir nombres para **resolver ambigüedades** mediante la anotación `@Qualifier`
 - Soporta inyección de **recursos externos**, e.g. EntityManager mediante la anotación `@PersistenceContext`

Ejemplo

```
@Controller
public class ClienteController {
```

```
    @Autowired
    @Qualifier("DAOMap")
    private ClienteDAO clienteDAO;
    13
```

o
también

```
@Autowired
@Qualifier("DAOMap")
public void setClienteDAO (ClienteDAO dao) {
    this.clienteDAO=dao;
}
```

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Inyección de dependencias usando anotaciones

@Controller

@RequestMapping("/clientes")

```
public class ClienteController {
```

@Autowired //or **@Inject (JakartaEE)**

@Qualifier("clienteDAOMap")

```
    private ClienteDAO clienteDAO;
```

@Repository("clienteDAOMap")

```
public class ClienteDAOMap
    implements ClienteDAO {
```

//...

```
}
```

@Repository("clienteDAOJpa")

```
public class ClienteDAOJpa
    implements ClienteDAO{
```

@PersistenceContext

```
private EntityManager em;
```

//...

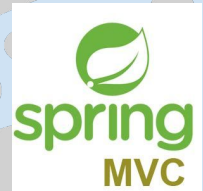
```
}
```

@Autowired localiza un componente en los paquetes indicados que sea compatible con el atributo asociado. Si hay varias opciones se resuelve mediante **@Qualifier** la opción válida.



Spring MVC

- Módulo de Spring para Implementación MVC en servidor
- Basado en un **controlador principal** (o *Front Controller*) que gestiona todas las peticiones (enrutamiento): **DispatcherServlet**
- El controlador principal trata de localizar los **controladores que atiendan cada petición** usando diferentes mecanismos (*Handler Mappings*)
 - Por defecto *RequestMappingHandlerMapping*
 - Los Controladores son clases anotadas: **@Controller**
 - Enrutamiento de peticiones a métodos:
 - **@RequestMapping(value="url", method="...")**
 - **@GetMapping(url)**, **@PostMapping(url)**...
- **El controlador selecciona la vista** independientemente de la tecnología empleada:
 - *JSP, Facelets, FreeMaker, Velocity, Thymeleaf, etc.*

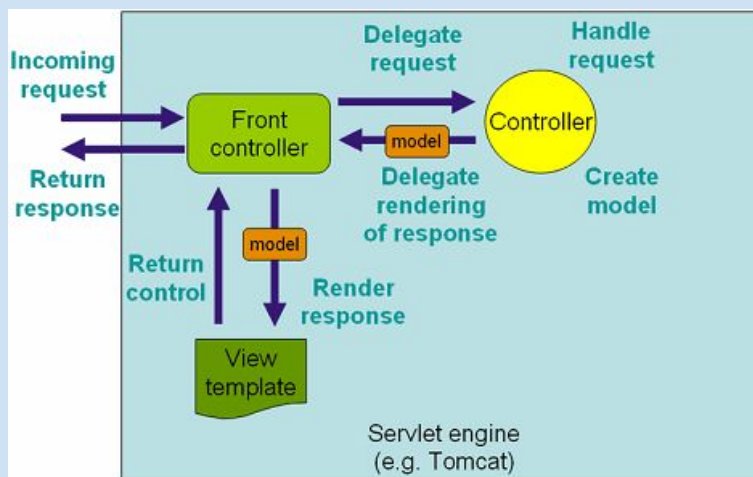


15

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Procesamiento de una Petición HTTP en SpringMVC



Fuente: Spring Framework Reference Documentation

16

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Inicialización de Spring Boot

- Consideraciones de configuración

- La anotación **@SpringBootApplication** se encarga de inicializar por defecto todos los recursos en dependencias del proyecto (starters), e.g. SpringMVC, JPA, etc.
- El **DispatcherServlet** de SpringMVC se lanza en Tomcat **como un Servlet más**
- Los **ficheros .jsp** deben albergarse en raíz **main/src/webapp**
- Algunos aspectos de la aplicación web pueden configurarse en fichero **webapp/WEB-INF/web.xml**

@SpringBootApplication

```
public class ClubSpringBootMvcApplication {

    // App starting but Spring context not available
    public static void main(String[] args) {
        SpringApplication.run( ClubSpringBootMvcApplication.class,
                                args);
    }

    public CommandLineRunner initApp (
        @Qualifier("clienteDAOMap") ClienteDAO clienteDAO) {
        //Spring application context fully started, i.e. CDI available
        return (args) -> {

            logger.info("Creando clientes de prueba");

            clienteDAO.crea(
                new Cliente(0, "Paco López", "11111111-A", false) );
            clienteDAO.crea(
                new Cliente(0, "María Jiménez", "22222222-B", true) );
            clienteDAO.crea(
                new Cliente(0, "Carlos García", "33333333-C", true) );

        };
    }
}
```

17

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Configuración de SpringMVC en Spring Boot

- Configuración "programática" en clases con anotación **@Configuration**

@Configuration

```
public class ClubSpringMvcConfig implements WebMvcConfigurer {
```

```
    @Override
    public void addViewControllers ( ViewControllerRegistry registry ) {
        //Define routes to simple views
        registry.addViewController("/login").setViewName("login"); //e.g. /WEB-INF/jsp/login.jsp
        registry.addViewController("/").setViewName("redirect:index.html"); //src/main/webapp/index.html
    }
}
```

- Configuraciones en fichero de propiedades: **application.properties**

- Ubicación **/src/main/resources**

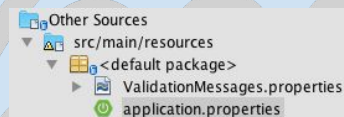
```
## JSP views location
```

```
spring.mvc.view.prefix=/WEB-INF/jsp/
```

```
spring.mvc.view.suffix=.jsp
```

- Errores personalizados en WEB-INF/jsp/error

- e.g. **WEB-INF/jsp/error/404.jsp**



18

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



SpringMVC

Controladores

Controladores Web en SpringMVC

- POJOs que atienden las peticiones HTTP
- **El framework se encarga de:**
 - Instanciar el objeto controlador (*singletons* como ocurría con los servlets)
 - Inyectar dependencias. e.g. DAOs, atributos, etc.
 - Asociar *rutas web* a métodos (*routing*) según el tipo de petición HTTP
 - Pasar (inyectar) parámetros (conversión de tipos y validación) de la petición web al método
 - Delegar/redirigir la petición a la vista seleccionada
- **Funciones del controlador (desarrollador)**
 - Interactuar con el Modelo (servicios, DAOs, etc.)
 - Pasar datos a la vista (modelo de vista)
 - Identificar a la vista que generará la salida html

Ejemplo de controlador web

```
package daw.club;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

@Controller

@RequestMapping("/clientes")

```
public class ClientesController {
```

@Autowired //or **@Inject**

```
ClienteDAO clienteDAO;
```

//Model DAO reference injected

@RequestMapping(value="/borra", method=GET) // url: /clientes/borra?id=5

```
public String borra ( @RequestParam(value="id", defaultValue="0") Integer id) {
```

```
    clienteDAO.borra(id);
```

//Interact with model

```
    return "redirect:listado";
```

//Redirect /clientes/listado

```
} //Other controller methods/mappings...
```

```
}
```

21

Ejemplo completo en

<https://github.com/jrbalsas/dawClubSpringBootMVC>

Recuperación de información de la petición

- Los parámetros de la petición se convierten (inyectan) a **parámetros anotados** del método que la atiende.
- **Anotaciones en parámetros de métodos**
 - **@RequestParam(value=,required=,defaultValue=)**
 - Parámetros de la forma param=valor (*query-string* o *body* si POST, e.g. de formulario)
 - **@PathVariable**
 - Parámetros en pathInfo de la URL e.g. /clientes/visualiza/{id}
 - **@CookieValue("cookie")**
 - Valor asociado a una cookie del navegador
 - **@HeaderParam("encabezado")**, encabezado de petición
- **Atributos de sesión**
 - Definidos en anotación de clase del controlador **@SessionValue("atributo1",...)**
 - Los atributos indicados que se almacenen en el modelo (ModelMap) se almacenan en la sesión

22

Selección de vista JSP/JSTL

• Enrutado directo hacia vistas

@Configuration

```
public class ClubSpringMvcConfig implements WebMvcConfigurer {
    //...
    @Override
    public void addViewControllers ( ViewControllerRegistry registry ) {
        //Define routes to simple views
        registry.addViewController("/login").setViewName("login"); // /WEB-INF/jsp/login.jsp

        registry.addViewController("").setViewName("redirect:/index.html"); //página por defecto
        //Other direct routes
    }
}
```

La ubicación y prefijo de las vistas se determina en la configuración de SpringMVC (viewResolver)

• Selección por método de controlador

```
@RequestMapping(value="/visualiza", method=GET)
public String visualiza (@RequestParam("id") Integer id ) { // url: /clientes/visualiza?id=5
    //request processing
    return "clientes/visualiza"; //Select view, /WEB-INF/jsp/clientes/visualiza.jsp
}
```

• Modificadores de vista

- "redirect:url", fuerza una redirección del navegador
 - e.g. implementación de patrón Post-Redirect-Get
 - return "redirect:listado"
- "forward:url", transfiere la petición a otra url diferente (por defecto)

23

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Envío información a la vista

- El controlador la coloca en un **objeto ModelMap** (hashMap) que se envía a los métodos que lo soliciten (inyección)

@Controller

@RequestMapping("/clientes")

```
public class ClientesController {
```

@ModelAttribute

```
private void configView(HttpServletRequest request, ModelMap model) {
    model.addAttribute("srvUrl", request.getContextPath()+
        request.getServletPath()+"/clientes");
    model.addAttribute("imgUrl", request.getContextPath()+"/images");
}
```

Método común a todas las peticiones

@RequestMapping(value="{", method=RequestMethod.GET) //url: /clientes or /clientes/listado

```
public String listado ( ModelMap model ) {
    List<Cliente> clientes=clienteDAO.buscaTodos();
    model.addAttribute("clientes", clientes);
    return "clientes/listado"; //show form
}
```

24

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Ejemplo CRUD Entidad cliente (continuación)

```
@Controller
@RequestMapping("/clientes")
public class ClienteController {
    // param in queryString e.g. /clientes/visualiza?id=1
    @GetMapping( "/visualiza" )
    public String visualiza(@RequestParam(value="id",required=true)Integer id, ModelMap model ) {
        model.addAttribute("cliente",clienteDAO.buscald(id));
        return "clientes/visualiza";
    }
    //Sample RESTful version (param in path_info instead of in query string, e.g. /clientes/visualiza/1)
    @GetMapping( "/visualiza/{id}" )
    public String visualizald(@PathVariable Integer id, ModelMap model ) {
        model.addAttribute("cliente",clienteDAO.buscald(id));
        return "clientes/visualiza";
    }
    @GetMapping("/borra")
    public String borra(@RequestParam(value="id",required=true)Integer id, ModelMap model) {
        clienteDAO.borra(id);
        model.clear(); //prevents model info to be sent in query-string on redirect
        return "redirect:listado";
    }
}
```

25

SpringMVC

Gestión de formularios y validación en
servidor

Manejo de formularios

- Los formularios asocian controles a propiedades de un *bean* (*Backend-Bean*)
- Puede ser un *bean* específico del modelo o uno específico para el formulario (DTO)
- Etiquetas JSP especializadas **<form:etiqueta...>**
 - `<%@taglib prefix="form" uri="http://www.springframework.org/tags/form" %>`
 - `<form:form method="POST" modelAttribute="backendBean" >`
 - **modelAttribute** referencia al **bean** asociado en el modelo de vista (*ModelMap*) enviado por el controlador.
 - Las propiedades del bean se asocian con controles del formulario con su atributo **path="propiedad"**
 - e.g. `<form:input path="propiedad" />`
- Al procesar el envío (POST), el controlador puede conocer si se han producido errores (conversión o validación) en la asignación de propiedades o generar errores personalizados

27

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Ejemplo de formulario

/WEB-INF/jsp/cliente/crea.jsp

```
<%@taglib prefix="form" uri="http://www.springframework.org/tags/form" %>
<!DOCTYPE html>
<html lang="es">
  <head>
    <title>Alta Cliente</title>
  </head>
  <body>
    <h1>Alta Cliente</h1>
    <form:errors path="cliente.*)" cssClass="error" element="div" />

    <form:form method="POST" modelAttribute="cliente" >
      <form:label path="nombre" >Nombre:</form:label><form:input path="nombre" />
      <form:errors cssClass="error" path="nombre"/><br>
      <form:label path="dni">DNI:</form:label><form:input path="dni" />
      <form:errors cssClass="error" path="dni"/><br>
      <form:label path="socio">Socio:</form:label><form:checkbox path="socio"/><br>
      <input name="enviar" type="Submit" value="Guardar">
      <input name="enviar" type="Reset" value="Limpiar">
    </form:form>
  </body>
</html>
```

La dirección de envío (**action**) es la del controlador que genera el formulario, e.g. /clientes/alta

28

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Ejemplo control de formulario

```
@GetMapping( "/crea" )
public String creaForm( ModelMap model ) {
    model.addAttribute("cliente",new Cliente());
    return "clientes/crea";    //render form WEB-INF/jsp/clientes/crea.jsp
}
```

1

@ModelAttribute asocia los parámetros del formulario con las propiedades de un bean

```
@PostMapping( "/crea" )
public String crea( @ModelAttribute("cliente") @Valid Cliente c,
                   BindingResult result, ModelMap model) {
    String view="redirect:listado"; //PRG → clientes/listado
    if ( ! result.hasErrors() ) {
        clienteDAO.crea(c);
        model.clear();
    }else {
        view="clientes/crea"; //Show error, and ask for correct data
    }
    return view;
}
```

2

@Valid Activa la validación "automática" de valores asociados a las propiedades del bean

BindingResult contiene los errores de conversión y validación de los controles del formulario a las propiedades del bean
DEBE IR JUSTO DESPUÉS DEL BEAN QUE SE VALIDA

Validación de valores asociados a propiedades de beans

- Los beans marcados con **@Valid** asociados a formularios se validan de dos formas:
 - Clases que implementan la interfaz **Validator** asociadas al controlador (específico de Spring) y métodos de validación estándar en **ValidationUtils** [+info](#).

```
public class ClienteValidator implements Validator {

    public boolean supports(Class clazz) {
        return Cliente.class.equals(clazz);
    }

    public void validate(Object obj, Errors e) {
        ValidationUtils.rejectIfEmpty( e, "nombre", "El nombre no puede estar vacío");
    }

}
```

- Soporte **Bean Validación** usando especificación **Jakarta Bean Validation**

Mensajes de error de validación

- Bean validation dispone de **mensajes de error predeterminados**
- **Mensajes de error personalizados:**
 - **Errores de Bean Validation**
 - **Atributo message=""** en restricción del bean
 - `@Min(value=18, message="Sólo se admiten mayores de edad")`
 - Identificadores de error en fichero `ValidationMessages.properties`
 - `@Min(value=18, message="{cliente.edad.min}")`
 - **Otros errores** (e.g. errores de integridad en modelo)
 - `result.rejectValue("propiedad", "id_mensaje", "Texto op. mensaje");`
 - e.g. `result.rejectValue("dni", "error.cliente.dniduplicado");`
- **Visualización errores en la vista:**
 - `<form:errors path="propiedad" cssClass="estilo">`
 - Muestra los errores asociados a una propiedad
 - `<form:errors path="bean.*" cssClass="estilo">`
 - Muestra todos los errores en propiedades del bean especificado

31

Ejemplo CRUD Entidad cliente (continuación)

```
@Controller
@RequestMapping( "/clientes" )
public class ClienteController {
    //...
    @GetMapping( "/edita" ) // /clientes/edita?id=1
    public String editaForm(@RequestParam(value="id",defaultValue="0")Integer id,
                           ModelMap model ) {
        model.addAttribute("cliente", clienteDAO.buscald(id));
        return "clientes/edita"; //→show WEB-INF/jsp/clientes/edita.jsp
    }
    @PostMapping( "/edita" ) // /clientes/edita?id=1
    public String edita(@ModelAttribute("cliente") @Valid Cliente cliente, BindingResult result,
                       ModelMap model) {
        String view="redirect:visualiza";
        if ( !result.hasErrors() ) {
            clienteDAO.guarda(cliente);
            model.clear();
            view=view+cliente.getId(); //post-redirect-get (e.g. /clientes/visualiza/1 )
        } else {
            view="clientes/edita"; //ask for correct data
        }
        return view;
    }
}
```

32

Spring Boot

Otros aspectos de configuración

Configuración JPA

- **Consideraciones:**

- SpringBoot utiliza su **propio pool de conexiones (Hikari)**. No se utiliza la definición JakartaEE del ~~DataSource en web.xml~~
- El **starter spring-boot-starter-data-jpa**:
 - *hibernate* como implementación de JPA
 - No utiliza ~~persistence.xml~~ para configurar la PU
 - Configura automáticamente una PU a partir de un datasource configurado
 - Fichero *resources/import.sql* para cargar datos de prueba (*insert*)

- Configuración en *resources/application.properties*

```
## Spring-boot own datasource initialization
spring.datasource.url=jdbc:h2:~/h2db/dawclub;AUTO_SERVER=TRUE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=dawuser
spring.datasource.password=dawuser
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
#Enable JPA debug and create data tables after starting app
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=create
```

Conclusiones

- Los **frameworks de aplicaciones web** aúnan principios de diseño, patrones y buenas prácticas en la implementación de aplicaciones web que facilitan su desarrollo
- A diferencia del enfoque de JEE basado en especificaciones, **los frameworks utilizan su propias APIs** y mecanismos de programación, lo que puede dificultar su migración a otras implementaciones
- **SpringMVC** es un módulo del **framework Spring** que facilita la creación de aplicaciones web en Java que siguen el modelo MVC
- **SpringBoot** es un framework para la creación y configuración rápida de aplicaciones basadas en Spring
- Spring puede utilizarse de forma exclusiva o **combinarse con otros elementos de JEE** a la hora de crear aplicaciones web basadas en Java.

Bibliografía

- Spring Framework Reference Documentation. VMWare, 2025. Disponible en <https://spring.io/projects/spring-framework>
- SpringMVC, in *Spring Framework Reference Documentation*. VMWare, 2025. Disponible en <https://docs.spring.io/spring/docs/current/spring-framework-reference/web.html>
- Wheeler, Willie; White, J. (2013) *Spring in Practice*. Safari. Disponible en [[Recurso Electrónico UJA](#)]
- Spring Boot. VMWare, 2025. Disponible en <https://spring.io/projects/spring-boot>

Lecturas Complementarias

- Spring JSP Tag library, in *Spring Framework Documentation*. VMWare, 2024. Disponible en <https://docs.spring.io/spring/docs/current/spring-framework-reference/web.html#mvc-view-jsp-tags>

Apéndice

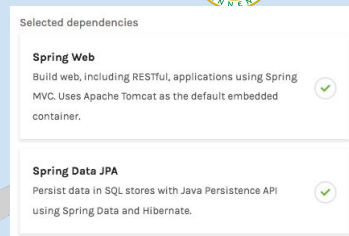
Despliegue de aplicación SpringBoot en servidor JakartaEE

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Inicialización de un proyecto Spring Boot para Servidor Jakarta

- <https://start.spring.io/>
 - Seleccionar tipo de proyecto maven, dependencias y tipo WAR
 - Plugin en IntelliJ para generar el proyecto
- **Despliegue en Payara**
 - Eliminar servidor embebido en pom.xml
 - Añadir dependencias JEE
 - Crear carpeta **src/main/webapp** para contenidos JEE, e.g. /WEB-INF/web.xml



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.3.3</version>
<relativePath/>
</parent>
```

```
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
<exclusions>
<exclusion> <!-- Embedded server not used -->
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-tomcat</artifactId>
</exclusion>
</exclusions>
</dependency>
```

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

<!-- Deployment in JEE Application Server -->

```
<dependency>
<groupId>jakarta.platform</groupId>
<artifactId>jakarta.jakartaee-web-api</artifactId>
<version>10.0.0</version>
<scope>provided</scope>
</dependency>
```

```
...
</dependencies>
```



Inicialización de Spring Boot

• Consideraciones de configuración

- Inicializar el **DispatcherServlet** de SpringMVC como un **Servlet** más
- Permite que el **servidor JEE** dé **acceso directo a otros recursos** en la aplicación: Servlets, ficheros JSP/JSF y recursos estáticos
- La **anotación @SpringBootApplication** se encarga de inicializar por defecto todos los recursos en dependencias del proyecto (starters), e.g. SpringMVC, JPA, etc.

@SpringBootApplication

```
public class ClubSpringBootMvcApplication extends SpringBootServletInitializer {

    // Configuración de SpringMVC al desplegar en servidor JakartaEE
    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        return application.sources( ClubSpringBootMvcApplication.class );
    }

    // punto de inicio para ejecución en línea de comandos
    public static void main(String[] args) {
        SpringApplication.run( ClubSpringBootMvcApplication.class, args);
    }
}
```

39

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025



Configuración JPA

• Consideraciones:

- Utilizaremos el **DataSource** y **Pool de conexiones** gestionado por el **servidor Jakarta** y definido en web.xml
- El **starter spring-boot-starter-data-jpa**:
 - *hibernate* como implementación de JPA
 - No utiliza ~~persistence.xml~~ para configurar la PU
 - Configura automáticamente una PU a partir de un datasource configurado
 - Fichero *resources/import.sql* para cargar datos de prueba (*insert*)

• Configuración en **application.properties**

```
# JNDI DataSource defined in web.xml
spring.datasource.jndi-name=java:global/jdbc/gestClub

# hibernate configure options
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=create // Auto create tables for Entities
```

40

Desarrollo de Aplicaciones Web <http://tiny.cc/dawuja>. 2025

